

SIMATS ENGINEERING

CO5 ASSESSMENT TOOL 2

NAME : JAIVANT.S

REG NO : 192311326

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE : CSA0613

Q13. Graph Isomorphism Problem — Backtracking

Problem Statement

Given two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$, determine whether they are **isomorphic**.

Two graphs are isomorphic if there exists a **one-to-one mapping** between their vertices such that adjacency relationships are preserved.

Input

- Graph G_1 with n vertices and edge set E_1
- Graph G_2 with n vertices and edge set E_2

Output

- TRUE — if the graphs are isomorphic.
 - FALSE — otherwise.
-

1. Conditions for Isomorphism

Before performing backtracking, check necessary conditions:

1. Both graphs must have the same number of vertices.
2. Both graphs must have the same number of edges.
3. Corresponding vertices must have the same degree.
4. The final mapping must preserve every edge relationship.

If any condition fails, return FALSE immediately.

2. Backtracking Approach

The algorithm constructs a mapping between vertices of the two graphs one vertex at a time.

For each vertex in G_1 :

1. Try mapping it to every unused vertex in G_2 .

2. Check whether the mapping satisfies the degree and adjacency constraints.
 3. If valid, recursively map the next vertex.
 4. If the mapping leads to a contradiction, **backtrack** and try another vertex.
-

3. Pseudocode

GraphIsomorphic($G1, G2$)

Input:

$G1 = (V1, E1)$

$G2 = (V2, E2)$

Output:

TRUE if $G1$ and $G2$ are isomorphic

FALSE otherwise

1. $n1 \leftarrow$ number of vertices in $G1$
2. $n2 \leftarrow$ number of vertices in $G2$
3. if $n1 \neq n2$:
4. return FALSE
5. if number of edges in $G1 \neq$ number of edges in $G2$:
6. return FALSE
7. if sorted degrees of $G1 \neq$ sorted degrees of $G2$:
8. return FALSE
9. mapping $\leftarrow$ empty array
10. used $\leftarrow$ array of $n2$ FALSE values
11. return Backtrack(0)

Backtrack(i)

12. if $i == n1$:
13. return TRUE
14. $u \leftarrow V1[i]$
15. for each vertex v in $V2$:

```

16.  if used[v] == TRUE:
17.      continue

18.  if degree(u) ≠ degree(v):
19.      continue

20.  valid ← TRUE

21.  for j = 0 to i-1:
22.      u2 ← V1[j]
23.      v2 ← mapping[j]

24.      if Edge(G1, u, u2) ≠ Edge(G2, v, v2):
25.          valid ← FALSE
26.          break

27.  if valid == TRUE:
28.      mapping[i] ← v
29.      used[v] ← TRUE

30.      if Backtrack(i + 1) == TRUE:
31.          return TRUE

32.      used[v] ← FALSE
33.      mapping[i] ← NULL

34. return FALSE

```

4. Constraint Checking

The important constraint is:

$$(u, u') \in E_1 \Leftrightarrow (m, \text{apping}[u] \text{mapping}[u']) \in E_2$$

This means an edge between two vertices in G_1 must correspond to an edge between their mapped vertices in G_2 .

If this condition fails, the current mapping is immediately rejected.

5. Pruning Techniques

The following techniques reduce unnecessary mappings:

- **Degree pruning:** Only map vertices having the same degree.

- **Used-vertex pruning:** A vertex in G_2 cannot be mapped more than once.
- **Edge-consistency pruning:** Reject a mapping as soon as an adjacency mismatch is detected.
- **Initial graph checks:** Different numbers of vertices, edges, or degree sequences immediately return FALSE.
- **Recursive backtracking:** Undo an invalid mapping and try another candidate.

These techniques significantly reduce the number of mappings compared with blindly checking all $n!$ permutations.

6. Complexity Analysis

For n vertices, there can be up to:

$$n!$$

possible vertex mappings.

Therefore, the worst-case time complexity is approximately:

$$O(n! \times n)$$

when adjacency constraints are checked during each mapping.

The space complexity is:

$$O(n)$$

for the mapping array, used array, and recursion stack, excluding the graph representation.

7. Example

Suppose:

$$G_1: V = \{1, 2, 3\}$$

and

$$G_2: V = \{A, B, C\}$$

If the mapping is:

$$1 \rightarrow A, 2 \rightarrow B, 3 \rightarrow C$$

and every edge in G_1 corresponds exactly to an edge in G_2 , the algorithm returns:

TRUE

If no valid vertex mapping preserves all edges, it returns:

FALSE

Conclusion

The backtracking algorithm systematically explores possible one-to-one vertex mappings between two graphs. **Degree checking, edge consistency, and used-vertex constraints** prune invalid mappings early. If a complete mapping preserving all adjacency relationships is found, the graphs are isomorphic; otherwise, they are not. The factorial worst-case complexity demonstrates why graph isomorphism can become computationally expensive for large graphs.
